Automated Website Deployment on AWS EC2

with Apache and a GitHub Actions CI/CD Pipeline

Project: ec2-cicd-website **Author:** Arun **Date:** 24 October 2026

Region: Asia Pacific (Mumbai), ap-south-1 **Live URL:** http://15.207.99.113

1. Summary

This project hosts a static website on an Amazon EC2 instance, served by the Apache HTTP Server, and deploys it automatically with a GitHub Actions CI/CD pipeline. After a change to the HTML file is pushed to GitHub, the live website updates on its own, with no manual file copying. The test change replaced the original CloudLaunch page with the Lumen page, and the live site updated automatically.

Component	Role
Amazon EC2 (t3.micro, Amazon Linux 2023)	Virtual server that hosts the website
Apache HTTP Server (httpd 2.4.69)	Web server that serves the site over HTTP, port 80
Security Group	Firewall controlling inbound traffic
GitHub repository	Version-controlled website files and workflow configuration
GitHub Actions + deploy.yml	Automated pipeline that deploys the site to EC2

Workflow: Code, Push, Deploy. You update the HTML file, commit it to GitHub, and GitHub Actions transfers it to EC2.

2. Architecture

Developer → git push → GitHub repository → GitHub Actions (deploy.yml) → EC2 instance → Apache (httpd) serves /var/www/html → Visitors over HTTP port 80 at http://15.207.99.113

3. EC2 Instance

The instance **CloudLaunch-website** (`i-0182fd2bd6b539760`) is a t3.micro running in availability zone ap-south-1b.

Property	Value
Instance state	Running
Public IPv4	15.207.99.113
Public DNS	ec2-15-207-99-113.ap-south-1.compute.amazonaws.com
Private IPv4	172.31.9.66
VPC	vpc-09346f99d95f3f65d
Elastic IP	None (see recommendations)

A second instance, snartgrid, exists in the account but is stopped.

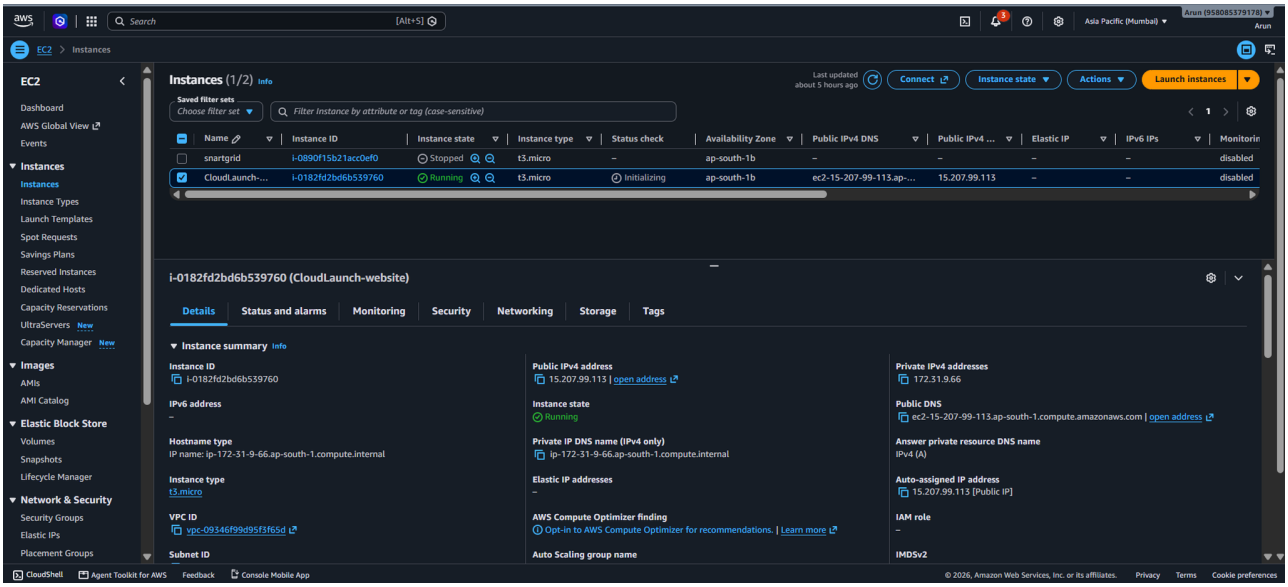


Figure 1: EC2 Instances console showing CloudLaunch-website running with public IP 15.207.99.113

4. Apache Web Server

What is Apache?

In an Amazon Web Services (AWS) environment, Apache (specifically the Apache HTTP Server) is used as a web server to host and deliver websites and web applications from an Amazon EC2 instance.

While Amazon EC2 provides the raw computing power (the virtual server), Apache provides the software layer needed to handle web traffic, process HTTP requests, and serve files to a user's web browser.

Setup and verification

The Linux terminal session on the instance shows that Apache is installed and running:

- `sudo dnf install -y httpd` installed Apache. The package httpd-2.4.69-1.amzn2023.0.2 was already installed.
- `sudo systemctl enable --now httpd` enabled Apache and started it immediately.
- `sudo mkdir -p /var/www/html` created the web root, where Apache looks for site files.
- `ls -l /var/www/html/index.html` showed index.html (3340 bytes) in the web root.
- `sudo systemctl status httpd` reported active (running) since Sat 2026-10-24 10:56:32 UTC. Main PID is 27151, and the log reads "Server configured, listening on: port 80".

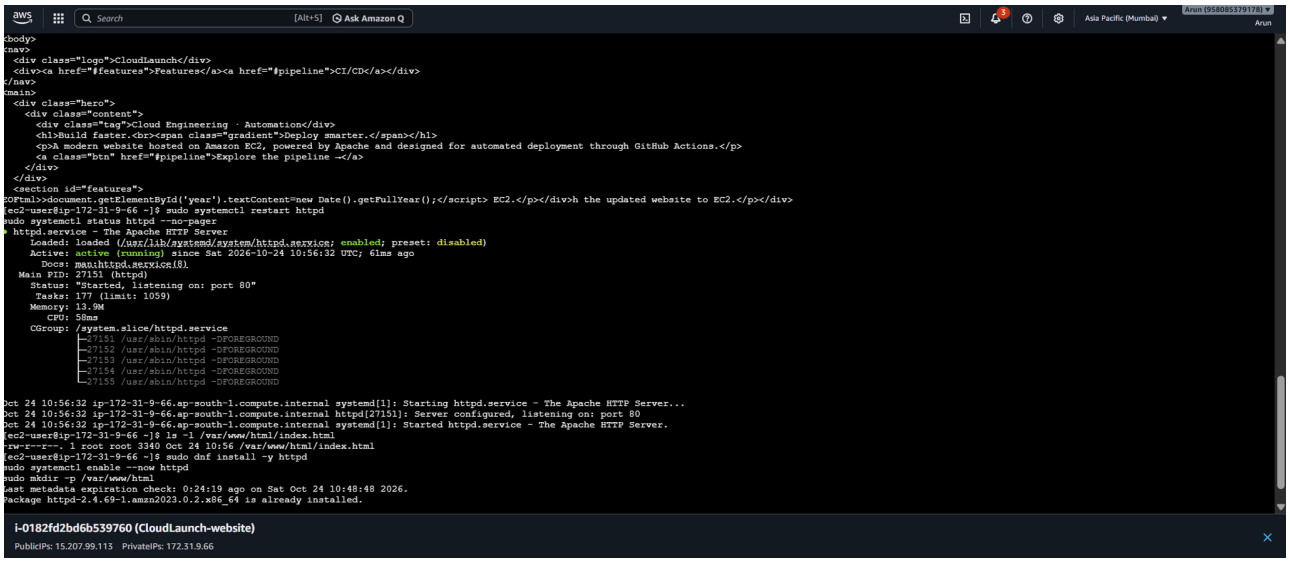


Figure 2: Apache (httpd) active and running on the Linux terminal, with index.html in /var/www/html

5. Security Group (Firewall)

Security group `sg-01a47a7af51bb0eb4` has three inbound rules:

#	Type	Port	Source	Purpose
1	SSH	TCP 22	Prefix list pl-0fa83cebf909345ca	Restricted SSH access (console connect)
2	HTTP	TCP 80	0.0.0.0/0 (Anywhere-IPv4)	Lets visitors reach the website
3	SSH	TCP 22	0.0.0.0/0 (Anywhere-IPv4)	Open SSH access

HTTP open to the world is required for a public website. AWS flags the open SSH rule with a console warning (see Section 9).

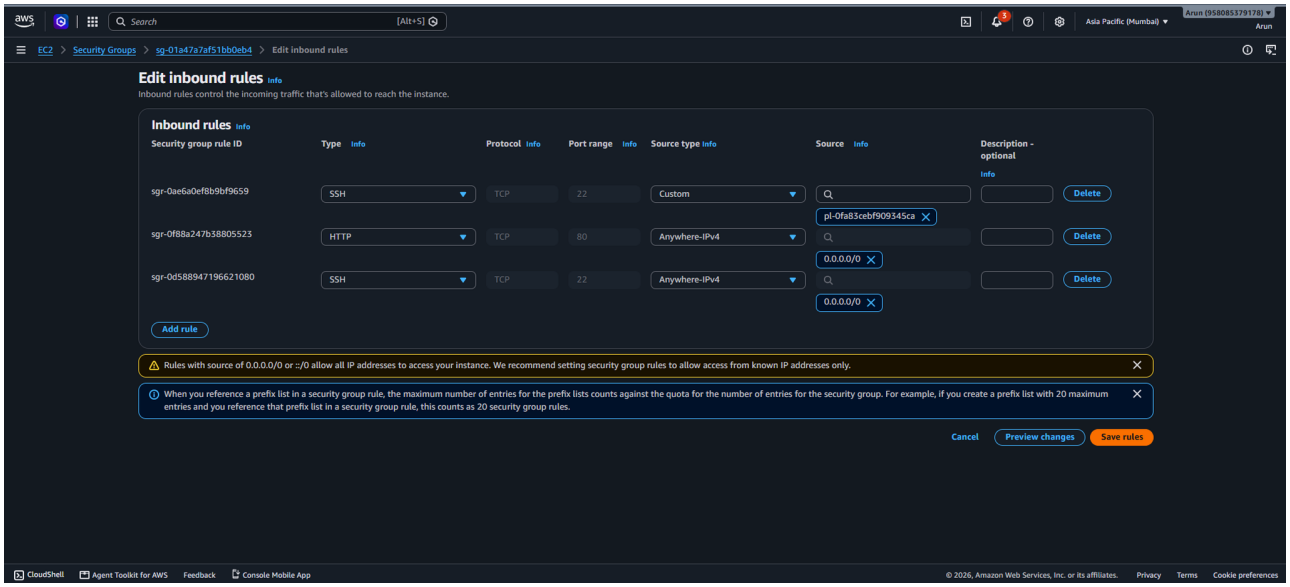


Figure 3: Security group inbound rules (SSH, HTTP, SSH) with the AWS 0.0.0.0/0 warning

6. deploy.yml: the GitHub Actions workflow

deploy.yml is the configuration file that tells GitHub Actions how to deploy your website from GitHub to your AWS EC2 server automatically. Think of it as an instruction manual for your CI/CD pipeline.

deploy.yml is the automation recipe. Without it, you would have to manually copy your updated website files to EC2 every time you made a change.

It lives in the repository at `.github/workflows/deploy.yml`. The workflow is named "Deploy Website to EC2". It runs when you push to main and can also be started manually, as run #2 was.

Repository secrets used by the workflow

To connect to the server, the workflow needs the EC2 address and login details. These are not written in the repository. They are stored as encrypted **GitHub Actions repository secrets** (Settings > Secrets and variables > Actions). Three secrets are configured:

- EC2_HOST: the address of the EC2 server to deploy to.
- EC2_USER: the Linux user the workflow logs in as.
- EC2_SSH_KEY: the private SSH key used to authenticate.

Secrets are encrypted and their values are hidden after they are saved. Only the names and last-updated times are visible. Anyone with collaborator access can use them in workflows, but they are not passed to workflows triggered by a pull request from a fork. Keeping credentials here instead of in deploy.yml means they are never exposed in the repository code.

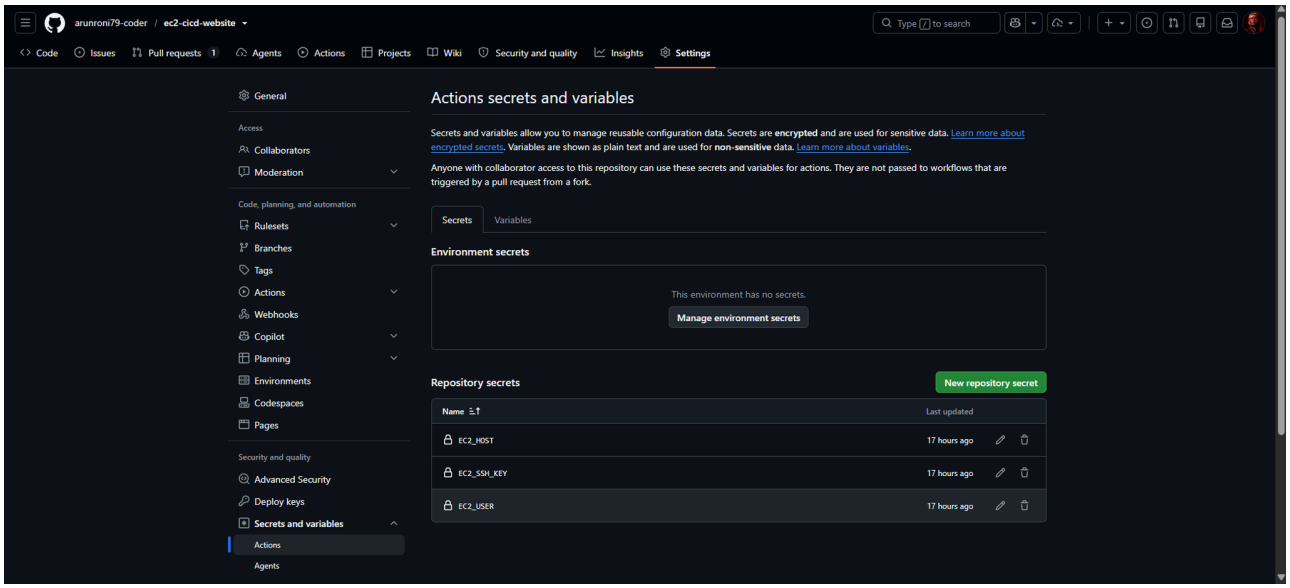


Figure 4: GitHub repository Actions secrets: EC2_HOST, EC2_SSH_KEY and EC2_USER

7. GitHub Actions run history

Repository: `arunroni79-coder/ec2-cicd-website`

Run	Workflow / message	Trigger	Branch	Result	Time taken	Time
#1	deploy.yml file	Push (ea6c0fa)	main	Failed	8s	16:40
Copilot #1	Running Copilot cloud agent	Copilot	copilot/fix-deploy-github-actions-job	Success	2m 4s	16:44
#2	Deploy Website to EC2	Manual run	main	Success	21s	16:54
#3	ec2 html file changed from cloudlaunch to lumen	Push (f61b85b)	main	Success	16s	17:31

What happened:

1. The first run failed, because the workflow needed correcting.
2. Copilot cloud agent was used on a fix branch to repair the deploy job.
3. After the fix, a manual run deployed the site successfully.
4. The final commit changed the HTML from CloudLaunch to Lumen. The workflow ran by itself in 16 seconds, which confirmed the pipeline works.

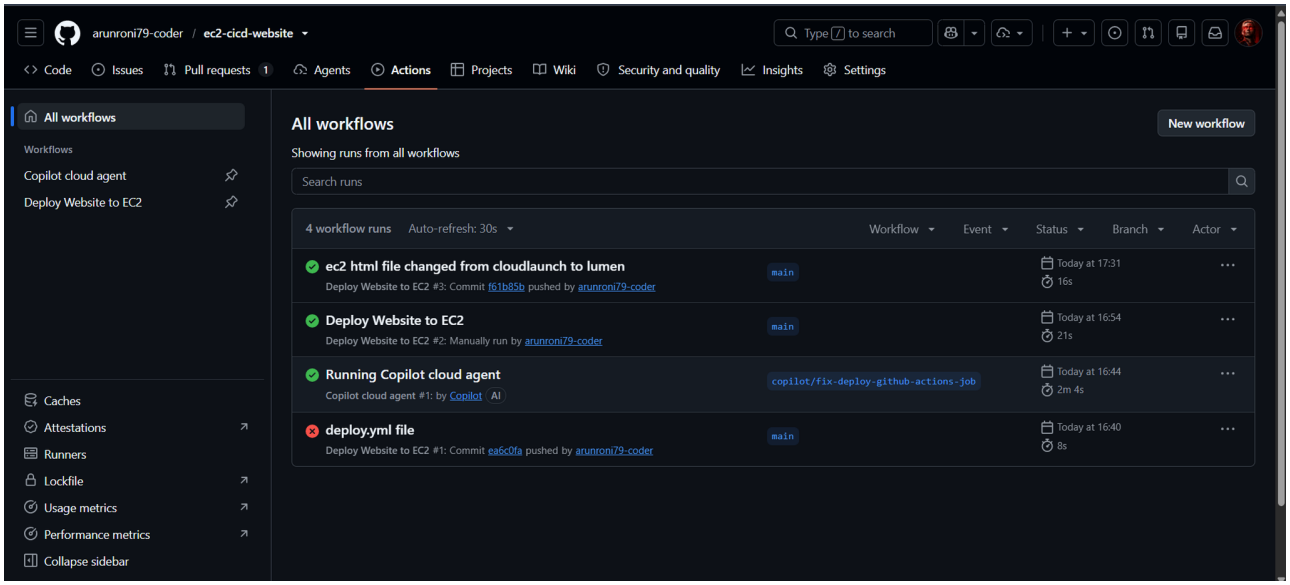


Figure 5: GitHub Actions "All workflows" page showing the four runs

8. Result: the website before and after

Before: CloudLaunch

The page was titled "Build faster. Deploy smarter." under the tag "Cloud Engineering · Automation". It included an "Explore the pipeline" button, three cards (Amazon EC2, GitHub, Automated CI/CD), and a three-step section "From commit to live website": 01 Code, 02 Push, 03 Deploy.

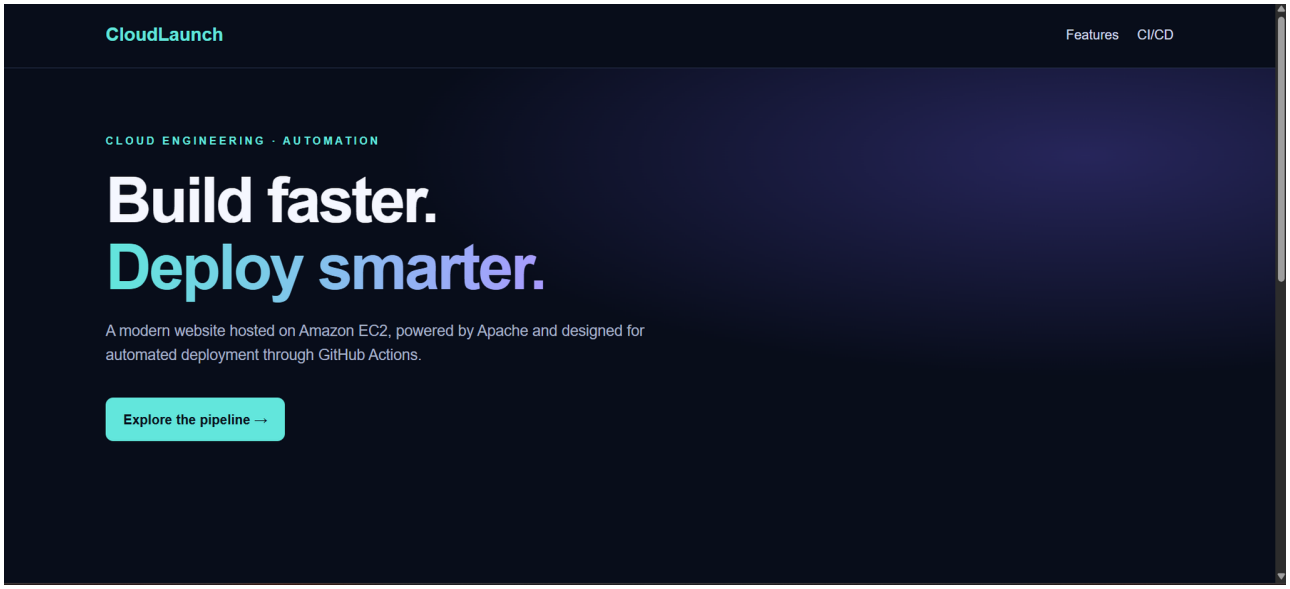


Figure 6: CloudLaunch website, top of the page

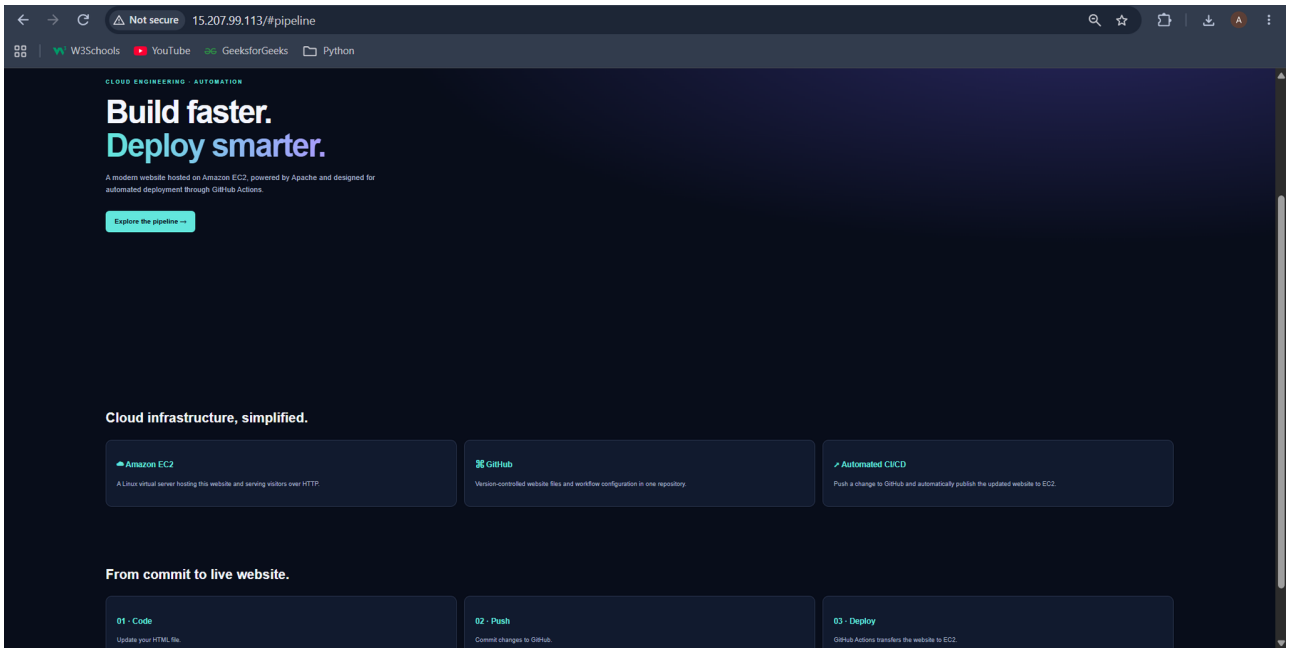


Figure 7: CloudLaunch website at 15.207.99.113/#pipeline, showing the EC2, GitHub and CI/CD cards and the Code, Push, Deploy steps

After: Lumen

After the push, the same URL showed the Lumen landing page. Its headline is "Focus better. Ship calmer." The page has a navigation bar, a task-board mockup, and statistics (48,000+ teams, 12M tasks, 99.9% uptime, 4.9 rating). It was updated with no manual server work.

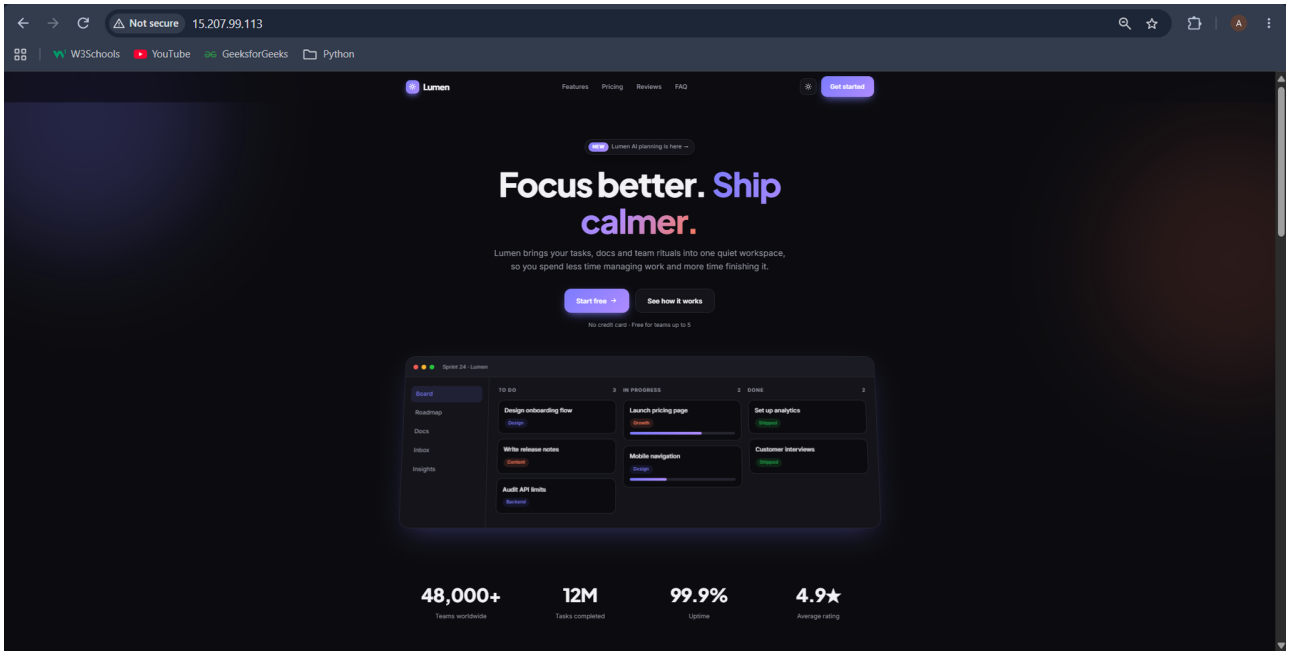


Figure 8: Lumen website at 15.207.99.113 after the automated deployment

9. Recommendations

1. **Restrict or remove the open SSH rule (0.0.0.0/0, port 22).** It lets any IP address attempt to log in. Limit it to known IPs. If the GitHub Actions runner needs SSH, consider a more secure method such as AWS Systems Manager instead of an open port.
2. **Attach an Elastic IP.** Without one, the public IP can change when the instance is stopped and started, which breaks the URL and any deployment settings that reference it.

3. **Use HTTPS.** The browser shows "Not secure" because the site is HTTP-only. Add a domain and a free certificate (for example Let's Encrypt).
4. **Keep credentials in GitHub Secrets.** This is already done for EC2_HOST, EC2_USER and EC2_SSH_KEY (Section 6). Rotate the SSH key periodically, and never commit it to the repository.
5. **Stop or terminate unused instances** such as snartgrid to avoid unnecessary costs.

10. Conclusion

The project shows a working end-to-end CI/CD flow. An Apache web server on Amazon EC2 serves the site, a security group controls access, and a GitHub Actions workflow defined in deploy.yml publishes every push to main to the live server in about 16 seconds. Changing the page from CloudLaunch to Lumen confirmed it works.